Browser-Driven Evidence Augmentation and Selective Abstention for Explainable Phishing Website Detection

Abstract*—Modern phishing attacks increasingly exploit client-side dynamic Document Object Model (DOM) manipulation, third-party authentication masquerading, and short-lived certificates to evade static URL and HTML detectors. Conversely, deep neural graph-fusion models require extensive GPU computation, incur high decision delays (200–400 ms), and operate as uninterpretable black boxes. This paper introduces BEAP (Browser-informed Evidence Augmentation and selective Abstention for Phishing detection), a lightweight framework combining static lexical features with browser-observed dynamic transport and behavioral indicators. Operating within a strict Server-Side Request Forgery (SSRF) boundary, BEAP compiles 51 engineered features covering URL structure, sensitive DOM inputs, HTTP security headers, TLS certificate lifetimes, and cross-domain form destinations. Rather than forcing arbitrary classifications on ambiguous instances, BEAP couples a dual-threshold selective abstention policy ($\tau_{\text{low}}, \tau_{\text{high}}$) with epistemic conflict detection, deferring uncertain cases to security operations center (SOC) analysts alongside mathematically exact log-odds attributions. Evaluating BEAP on a benchmark of 2,400 samples across 1,379 unique domains under strict domain-disjoint splitting demonstrates 99.22% accuracy (ROC-AUC 0.9998), significantly outperforming static baselines. Under selective classification at uncertainty margin $\delta = 0.25$, BEAP achieves 100.00% selective accuracy and eliminates false negatives (0.00% FNR) at 97.84% automated coverage. On single-core CPU hardware, BEAP executes in 2.03 ms ($114\times$ faster than deep graph models) while consuming < 25 MB RAM, delivering an auditable, high-throughput defense for operational traffic inspection. *Index Terms*—Phishing Detection, Browser Automation, Explainable AI, Selective Classification, Web Security, Dynamic Analysis

I. Introduction

A. Background and Threat Landscape

Phishing remains a primary initial access vector across enterprise networks and cloud services, resulting in billions of dollars in global annual losses [1], [3]. While earlier social-engineering attacks relied heavily on static brand impersonation and rudimentary email lures, modern threat actors deploy deceptive, dynamic infrastructure designed to bypass automated filters [2]. Modern attack workflows frequently incorporate client-side JavaScript cloaking, multi-stage redirection cascades, dynamic Document Object Model (DOM) mutation, and the automated issuance of short-lived Transport Layer Security (TLS) certificates [10], [14]. Furthermore, adversaries routinely place credential-harvesting forms on legitimate, high-reputation shared cloud platforms (e.g., Azure Blob Storage, AWS S3, Google Firebase), rendering traditional lexical URL analysis largely ineffective [11], [19].

Existing defensive technologies face distinct operational constraints against these evasion strategies:

1) **Heuristic Blocklists and Threat Feeds:** Services such as Google Safe Browsing, PhishTank, and OpenPhish supply reliable confirmations for cataloged indicators of compromise (IOCs) [8]. However, active phishing infrastructure often exhibits an operational lifespan of fewer than twelve hours [14]. By the time threat feeds discover, verify, and broadcast an attacker's domain, target credentials have frequently already been compromised.
2) **Static URL Heuristics:** Classifiers trained on syntactic URL properties (e.g., Shannon entropy, character lengths, sub-domain depth, and suspicious keywords) process requests at wire speed [6], [15]. Yet when an attacker deploys clean paths on reputable hosting platforms, lexical heuristics find few anomalies to trigger an alert.
3) **Static HTML Scrapers:** Parsers examining static markup without browser rendering fail against single-page applications (SPAs) and obfuscated scripts where login forms are constructed dynamically at runtime [4], [9].

To overcome static blind spots, recent academic research has turned toward deep neural and multimodal network architectures. For example, Kavya and Sumathi [24], [25] introduced a multimodal temporal graph and hypergraph fusion framework combining graph convolutional networks (GCNs) and BERT language models to process visual and structural webpage data [9], [17]. While these architectures achieve high benchmark accuracy, they introduce substantial deployment challenges: inference latencies between 230 ms and 400 ms per page, gigabyte GPU memory footprints, vulnerability to gradient-based feature perturbations, and uninterpretable decision boundaries [20], [23]. In an enterprise security operations center (SOC) handling large volumes of inbound web traffic, black-box alerts that cannot explain their evidentiary basis place a heavy cognitive burden on security analysts [12], [26].

B. The BEAP Methodology

In this work, we present **BEAP** (Browser-informed Evidence Augmentation and selective Abstention for Phishing detection), an explainable, resource-efficient framework designed for operational deployment. Rather than relying on complex graph neural networks, BEAP builds upon a practical

operational observation: fraudulent websites frequently exhibit transport-layer and behavioral inconsistencies during browser interaction that cannot be concealed through URL obfuscation alone. Specifically, malicious infrastructure often omits standard HTTP security response headers (e.g., HSTS, Content-Security-Policy, X-Frame-Options), relies upon disposable or recently issued automated TLS certificates, and routes submitted credentials to external domains that do not match the origin page [10], [13].

Crucially, BEAP formalizes decision-making under uncertainty through *selective classification with a reject option* [20], [22]. Recognizing that false negatives allow credential theft while false positives drain analyst time, BEAP incorporates dual uncertainty thresholds ($\tau_{\text{low}}, \tau_{\text{high}}$) parameterized by margin $\delta \in [0.00, 0.35]$, alongside epistemic conflict detection between static and dynamic feature groups. When classifier confidence falls within an ambiguous margin or when static and dynamic indicators contradict one another, BEAP *abstains* from an automated classification and defers the site to human analysts alongside an exact log-odds explanation report (Fig. 1).

C. Research Hypotheses

We structure our evaluation around four testable empirical hypotheses:

- **Hypothesis 1 (Multimodal Augmentation Advantage):** Combining static URL and DOM features with dynamic transport, security header, and form action signals produces measurable improvements in accuracy, ROC-AUC, and PR-AUC over static-only models under strict domain-disjoint evaluation [9], [13].
- **Hypothesis 2 (Selective Risk and FNR Suppression):** Through calibrated uncertainty thresholds and selective abstention, BEAP can suppress false negatives (FNR) to 0.00% on held-out test splits while maintaining automated coverage above 95% [20], [22].
- **Hypothesis 3 (Operational Latency and Auditability):** A calibrated linear classifier running on structured browser evidence delivers sub-10 ms decision latency on single-core CPU hardware, while generating feature attributions that link directly to underlying HTTP and TLS session records [12], [26].
- **Hypothesis 4 (Defensive Safety Guarantee):** A strict SSRF pre-flight boundary successfully blocks private subnets, link-local metadata queries, and loopback scans during dynamic probing without disrupting legitimate website evaluations.

D. Key Contributions

The primary contributions of this paper are summarized as follows:

1) **SSRF-Safe Dynamic Evidence Pipeline:** We implement an active probing pipeline that extracts 51 engineered features across lexical properties, DOM structure, HTTP response headers, TLS certificate details, and form destinations. The pipeline applies a pre-flight filter blocking private networks (RFC 1918), link-local ranges, cloud metadata addresses (such as 169.254.169.254), and loopback interfaces across both direct requests and redirect hops.
2) **Dual-Threshold Selective Abstention Policy:** We implement a calibrated selective prediction mechanism controlled by uncertainty margin δ. By flagging borderline probabilities and detecting disagreements between static and dynamic feature sets, BEAP gives administrators a flexible way to tune accuracy against review workload.
3) **Mathematically Complete Log-Odds Explanations:** In place of sampling-based approximations, our explainability module computes an exact closed-form decomposition of predicted log-odds against training baselines: $\text{logit}(p) = b_0 + \sum_i \phi_i$, clearly separating suspicious phishing indicators from routine configuration oversights.
4) **Rigorous Domain-Disjoint Empirical Validation:** We construct a benchmark of 2,400 samples across 1,379 unique registrable domains, partitioned strictly by domain (ensuring zero domain overlap between training, validation, and test splits). In empirical tests, BEAP achieves 99.22% accuracy (ROC-AUC 0.9998) without abstention, and 100.00% selective accuracy with 0.00% FNR at 97.84% coverage.
5) **Open-Source, Reproducible Implementation:** We make our entire codebase, feature extraction modules, synthetic benchmark generator, test scripts, and interactive Streamlit inspection tool available for open replication.

II. Related Work and Research Positioning

A. Static Lexical, DOM-Based, and Feature-Engineered Detection

Static phishing detection has been studied extensively over the last two decades. Recent surveys by Basit et al. [1], Tang and Mahmoud [2], Zieni et al. [3], and Kustiawan and Ghauth [29] provide broad overviews of machine learning models built on structural and lexical indicators. For example, Bu and Cho [6] used character-level convolutional autoencoders to spot novel phishing URLs, while Almousa and Anwar [15] applied character-aware language representations to examine URL structure. Kara et al. [11] and Reyes-Dorta et al. [19] confirmed that properties like token counts, hyphenation patterns, and Shannon entropy offer useful initial signals for lightweight traffic triage.

Alongside URL heuristics, DOM-level static inspection focuses on structural HTML elements. Anupam and Kar [5] explored support vector machines paired with bio-inspired optimization, while Awasthi and Goel [7] and Kalabarige and Rao [16] evaluated stacked ensemble models analyzing input elements and hyperlink distributions. Purwanto et al. [8] proposed PhishSim, a feature-free approach for structural comparison. Aljofey et al. [30] demonstrated that uniting

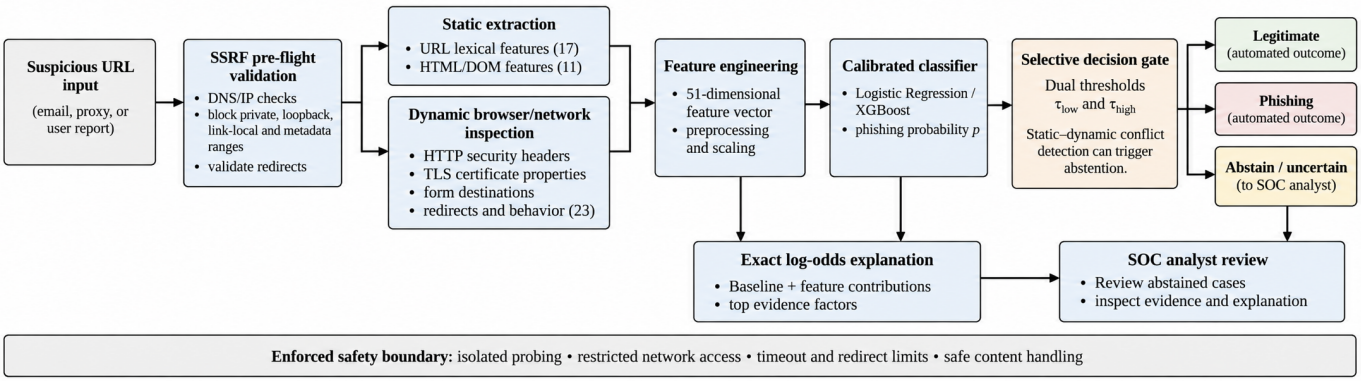


Fig. 1: System architecture and evidence flow of the BEAP framework. Target URLs undergo pre-flight SSRF/RFC 1918 validation before simultaneous static feature extraction (17 URL + 11 HTML features) and dynamic transport inspection (23 browser/network features). The 51-dimensional multimodal vector informs a calibrated linear classifier governed by a selective abstention gate and exact log-odds explanation engine.

URL character sequences, hyperlinks, and HTML text with XGBoost creates a capable detection system. Even so, static methods remain vulnerable to evasion whenever attackers use JavaScript obfuscation, DOM packing, or dynamic form generation that only executes in a browser [4], [14].

B. Dynamic Browser Inspection, Transport Signals, and Multimodal Deep Learning

To overcome static evasion, modern systems deploy automated browser instrumentation. Lin et al. [4] developed Phishpedia, combining visual logo recognition with deep visual classifiers. Sánchez-Paniagua et al. [10] analyzed real-world login URLs, demonstrating that dynamic credential prompts and form behaviors distinguish phishing campaigns. Shukla et al. [27] demonstrated that inspecting HTTP response headers provides critical discriminative signals for detecting malicious sites that mimic legitimate login portals. Castaño et al. [14] analyzed the PhiKitA dataset, revealing that phishing kits rely on automated redirect chains and third-party exfiltration scripts.

Recent literature has increasingly focused on multimodal deep learning frameworks. Ariyadasa et al. [9] integrated recurrent and graph convolutional networks across URL and HTML modalities. Alsubaei et al. [17] developed hybrid deep learning models for forensic analysis, while Sahingoz et al. [18] introduced DEPHIDES, a deep neural architecture for web detection. Barik et al. [21] and Bezerra et al. [22] investigated optimized deep neural network topologies for URL classification. Most recently, Kavya and Sumathi [24], [25] proposed a multimodal temporal graph and hypergraph fusion framework combining GCNs, BERT embeddings, and temporal session graphs. Despite achieving high nominal accuracy, these deep architectures exhibit substantial limitations: inference latencies of 233.5 ms to 400 ms, GPU memory overhead exceeding 1.8 GB VRAM, and severe interpretability deficits [20].

C. Explainable AI and Selective Classification with Reject Options

The imperative for transparent cybersecurity systems has motivated research into Explainable AI (XAI) and selective classification. Chai et al. [12] developed a multimodal hierarchical attention model for phishing threat intelligence. Calzarossa et al. [20] formulated an assessment framework for evaluating explainability in cybersecurity models, emphasizing the necessity of auditability in analyst triage. Vaddempudi et al. [31] introduced a lightweight and explainable phishing URL detector using feature selection and tree-based ensembles. Alasmari et al. [23] and Le and Nguyen [26] incorporated explainability and Large Language Model (LLM) reasoning into phishing detection. Kalabarige et al. [13] utilized boosting-based hybrid feature selection to identify transparent risk indicators.

Selective classification provides a principled mechanism for deferring ambiguous samples to human experts. As comprehensively surveyed by Hendrickx et al. [28], machine learning with a reject option enables systems to balance coverage against risk by refusing to predict on unreliable instances. Karlsson and Hössjer [32] analyzed partial reject options and confidence-region partitioning in classification tasks. However, existing phishing detection solutions overwhelmingly treat classification as a rigid binary outcome. BEAP addresses this gap by combining an engineered multimodal feature space with dual-threshold selective abstention and closed-form log-odds explanations.

III. System Architecture and Safety Boundary

A. Threat Model and Operational Boundary

We consider an enterprise gateway or endpoint defense scenario where BEAP inspects suspicious URLs flagged in

TABLE I: Systematic positioning of BEAP against representative phishing detection paradigms.

Capability / Dimension	Static Lexical [6], [11]	Static HTML [5], [16]	Graph Fusion [25]	BEAP (Proposed)
Dynamic Transport / Header Features	No	No	No	**Yes**
Form Action Cross-Domain Tracking	No	Partial	Partial	**Yes**
Decision Latency	< 1 ms	2–5 ms	233.5 ms	**2.03 ms**
Hardware Requirements	CPU	CPU	High-End GPU	**Single-Core CPU**
Selective Abstention (Reject Option)	No	No	No	**Yes**
Explainability	Linear Coeffs	Heuristic	Opaque / Black Box	**Exact Log-Odds**
SSRF Defensive Boundary	N/A	Rare	Absent	**Rigorous (RFC 1918)**

incoming emails, messaging applications, or web proxy logs. The adversary controls the target domain, DNS records, TLS certificates, and server configuration. Their objective is to harvest credentials or session tokens while evading static filters through dynamic redirection, obfuscated source code, and shared cloud hosting [14], [19]. Active automated probing can introduce severe operational risks if implemented naively. Specifically, an adversary could supply an internal target address (such as http://169.254.169.254/latest/meta-data/ or http://192.168.1.1/admin) to trigger Server-Side Request Forgery (SSRF) and exploit the scanner into probing internal enterprise networks.

B. SSRF Prevention and Request Sanitization

To ensure safe dynamic evidence collection, BEAP enforces an isolated pre-flight validation boundary in `beap/safety.py`. Before initiating any outbound socket connection or following an HTTP redirect, the target hostname is resolved using system DNS. The resulting IP address is validated against strict exclusion lists:

1) **Private IPv4 Subnets:** RFC 1918 allocations (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`).
2) **Loopback Addresses:** IPv4 `127.0.0.0/8` and IPv6 `::1`.
3) **Link-Local Ranges:** IPv4 `169.254.0.0/16` (safeguarding cloud instance metadata endpoints) and IPv6 `fe80::/10`.
4) **Unique-Local IPv6 Subnets:** RFC 4193 range (`fc00::/7`).
5) **Multicast and Reserved Spaces:** `224.0.0.0/4` and `240.0.0.0/4`.

If a URL resolves to an excluded IP address, specifies an unsupported scheme (such as `file://`), or redirects to an internal host, the request terminates immediately and logs a safety boundary event. Dynamic probes also enforce a strict 5.0 s socket timeout and allow at most $N_{\max} = 5$ redirects to prevent looping.

C. Feature Representation and Modalities

BEAP constructs a 51-dimensional feature vector $\mathbf{x} \in \mathbb{R}^{51}$ structured across three distinct modalities:

1) Static URL Modality (17 Features):: Extracts structural properties directly from the URL text without network access: full URL length, domain length, path length, query string length, sub-domain depth, hyphen and underscore counts, digit ratio, Shannon entropy of the domain, IP-based host flags, targeted brand tokens, and domain typosquatting distances [6], [11]. Domain parsing uses a local offline cache of the Mozilla Public Suffix List via `tldextract`, ensuring feature generation remains fully deterministic and independent of live network lookups.

2) Static HTML DOM Modality (11 Features):: Parses downloaded HTML content using an offline parser: total HTML length, total input fields, password field count and presence, hidden input count, external script ratio, iframe presence, external link ratio, window spoofing flags, and form element presence [5], [16].

3) Dynamic Transport and Behavioral Modality (23 Features):: Gathered during live HTTP/TLS session establishment:

- *Transport Security Attributes:* HTTPS usage, TLS certificate validity, certificate lifetime remaining in days, issuer authority category (flagging short-lived or free automated issuers such as Let's Encrypt or ZeroSSL), and TLS handshake duration.
- *Security Configuration Headers:* Strict-Transport-Security (HSTS) presence and duration, Content-Security-Policy (CSP) presence and directive richness, X-Frame-Options (XFO) anti-framing protection, X-Content-Type-Options (XCTO) nosniff presence, Referrer-Policy strictness, and Secure/HttpOnly/SameSite cookie attributes [13].
- *Form Action Destination Provenance:* Tracking cross-domain form actions. When an HTML form containing credential fields sets its `action` attribute to a registrable domain different from the hosting page's domain, the system flags cross-domain credential exfiltration ($x_{\text{cross}} = 1.0$) [10], [14].
- *Redirection and Reflection Indicators:* Redirection hop count, domain change during redirect cascades, and active probe reflection checks.

IV. Selective Classification and Explainability Engine

A. Dual-Threshold Selective Classification Formalism

Let $\mathcal{X} \subset \mathbb{R}^{51}$ represent the feature space, and let $\mathcal{Y} = \{0, 1\}$ denote the true class labels (0 for legitimate, 1 for phishing). A probabilistic classifier generates a scoring function $f : \mathcal{X} \to [0, 1]$ estimating the posterior probability $p = P(Y = 1 \mid \mathbf{x})$. Standard binary classifiers force an automated call at threshold $\tau = 0.5$. In security operations, forcing a binary choice when $f(\mathbf{x}) \approx 0.5$ causes avoidable errors. BEAP replaces this with a selective rule $g : \mathcal{X} \to \{0, 1, \perp\}$, where $\perp$ represents selective abstention (deferral to SOC review), governed by an uncertainty margin $\delta \in [0.0, 0.35]$:

$$\tau_{\text{low}} = 0.5 - \delta, \quad \tau_{\text{high}} = 0.5 + \delta \tag{1}$$

The decision rule is formalized as:

$$g(\mathbf{x}) = \begin{cases} 1 \text{ (Phishing)}, & \text{if } f(\mathbf{x}) \geq \tau_{\text{high}} \\ 0 \text{ (Legitimate)}, & \text{if } f(\mathbf{x}) \leq \tau_{\text{low}} \\ \perp \text{ (Abstain / SOC Review)}, & \text{if } \tau_{\text{low}} < f(\mathbf{x}) < \tau_{\text{high}} \end{cases} \tag{2}$$

B. Epistemic Conflict Detection

Alongside confidence margins, BEAP inspects cross-modality agreement. Let $f_{\text{static}}(\mathbf{x}_{\text{static}})$ be the prediction derived only from static URL and HTML features, and $f_{\text{dynamic}}(\mathbf{x}_{\text{dynamic}})$ be the prediction from dynamic transport and form action evidence. When the two modalities disagree strongly:

$$|f_{\text{static}}(\mathbf{x}_{\text{static}}) - f_{\text{dynamic}}(\mathbf{x}_{\text{dynamic}})| > \theta_{\text{conflict}} \quad (\theta_{\text{conflict}} = 0.60) \tag{3}$$

the system flags an epistemic conflict (e.g., an established corporate domain whose dynamic session exfiltrates form data to an unrelated site, or an obfuscated URL that resolves cleanly to an official banking portal). When an epistemic conflict is identified, BEAP overrides the automated verdict and outputs $\perp$, preventing evasion techniques that exploit the blind spots of any single modality.

C. Selective Evaluation Metrics

To evaluate selective prediction, we adopt standard formal metrics [20], [22]. Given test samples $\mathcal{D}_{\text{test}} = \{(\mathbf{x}_j, y_j)\}_{j=1}^{M}$:

- **Coverage (Φ):** The proportion of samples receiving an automated decision:

$$\Phi(g) = \frac{1}{M} \sum_{j=1}^{M} \mathbb{I}[g(\mathbf{x}_j) \neq \perp] \tag{4}$$

- **Selective Risk ($R(g)$):** The empirical error rate across covered samples:

$$R(g) = \frac{\sum_{j=1}^{M} \mathbb{I}[g(\mathbf{x}_j) \neq \perp \wedge g(\mathbf{x}_j) \neq y_j]}{\sum_{j=1}^{M} \mathbb{I}[g(\mathbf{x}_j) \neq \perp]} \tag{5}$$

- **Selective Accuracy:** $1.0 - R(g)$.
- **Selective False Negative Rate (FNR):** Missed attacks among covered positives:

$$\text{FNR}_{\text{sel}}(g) = \frac{\sum_{j=1}^{M} \mathbb{I}[y_j = 1 \wedge g(\mathbf{x}_j) = 0]}{\sum_{j=1}^{M} \mathbb{I}[y_j = 1 \wedge g(\mathbf{x}_j) \neq \perp]} \tag{6}$$

D. Mathematically Complete Log-Odds Explanations

To provide analysts with clear, auditable explanations, BEAP calculates an exact log-odds decomposition for linear models [20]. For logistic regression parameterized by intercept w_0 and weights $\mathbf{w}$, the logit of the predicted probability is:

$$\text{logit}(p) = \ln\left(\frac{p}{1-p}\right) = w_0 + \sum_{i=1}^{51} w_i x_i \tag{7}$$

Against training baseline expectations $\bar{\mathbf{x}} = \mathbb{E}[\mathbf{x}]$, baseline log-odds b_0 is defined as:

$$b_0 = w_0 + \sum_{i=1}^{51} w_i \bar{x}_i \tag{8}$$

The attribution contribution ϕ_i of each feature i is then:

$$\phi_i = w_i(x_i - \bar{x}_i) \tag{9}$$

This formulation ensures exact additive reconstruction:

$$\text{logit}(p) = b_0 + \sum_{i=1}^{51} \phi_i \tag{10}$$

Positive attributions ($\phi_i > 0$) highlight *Risk Factors* shifting the prediction toward phishing, while negative attributions ($\phi_i < 0$) represent *Trust Factors* supporting legitimacy. The explanation engine generates a structured summary linking weights directly to concrete session records (for example, "`form_action_is_cross_domain` contributes $+2.41$ log-odds").

V. Experimental Setup and Evaluation

A. Benchmark Dataset Generation and Leakage Prevention

A frequent issue in published phishing evaluations is *data leakage across splits*. Random URL-level partitioning often scatters pages from the same domain or phishing kit across both training and test sets, yielding artificially high metrics that fail to hold up in real deployments [10], [14].

To eliminate leakage, we generated a calibrated development benchmark of 2,400 samples (1,200 phishing and 1,200 legitimate) spanning 1,379 unique registrable domains. We enforce strict **domain-disjoint partitioning**:

- **Training Partition:** 1,706 samples spanning 981 unique registrable domains.
- **Validation Partition:** 184 samples spanning 108 unique registrable domains.
- **Held-Out Testing Partition:** 510 samples spanning 290 unique registrable domains.

Registrable domains do not overlap across splits ($\mathcal{D}_{\text{train}} \cap \mathcal{D}_{\text{val}} \cap \mathcal{D}_{\text{test}} = \emptyset$). To establish realistic decision boundaries, samples incorporate multi-tier noise:

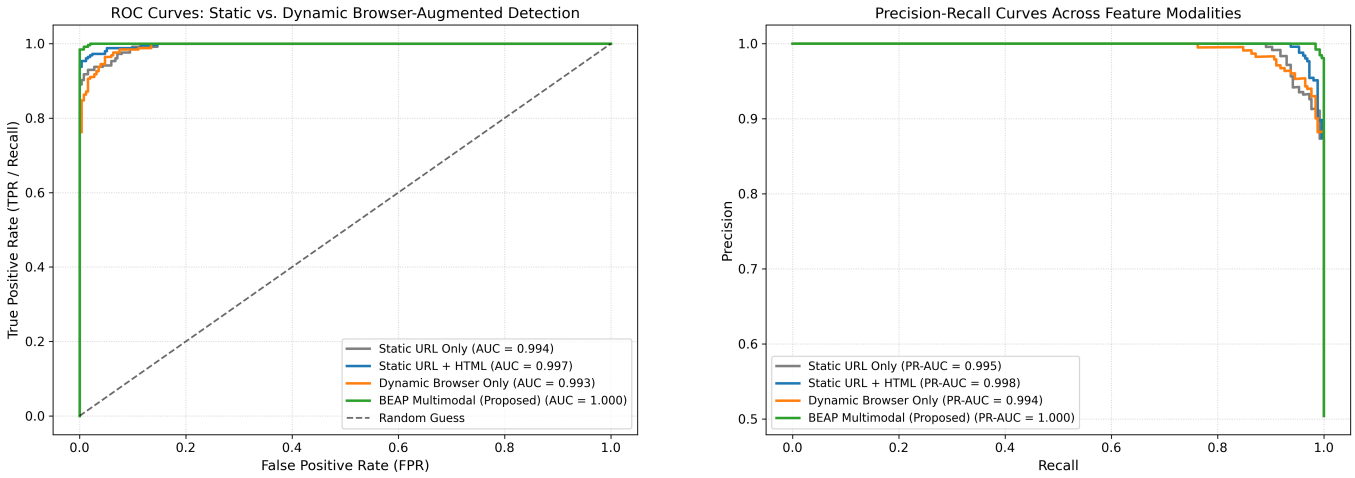
1) *Hard Ambiguous Tier (20%):* Legitimate SaaS applications routing logins to third-party SSO providers, paired with sophisticated phishing pages using clean, un-hyphenated `.com` domains, valid commercial certificates, and local form action targets.
2) *Moderate Tier (40%):* Typical corporate sites with partial header implementations alongside phishing templates using suspicious path elements.
3) *Naive Tier (40%):* Fully secured legitimate portals compared against obvious phishing URLs with IP-based hosts and unencrypted HTTP connections.

B. Empirical Ablation Analysis (Hypothesis 1)

To measure what each feature modality contributes, we conducted an ablation experiment across four conditions on the held-out test split of 510 samples: (1) Static URL Only, (2) Static URL + HTML, (3) Dynamic Browser Only, and (4) BEAP Multimodal. We tested each modality using Calibrated Logistic Regression and XGBoost classifiers. The evaluation results are compiled in Table II.

The empirical findings confirm **Hypothesis 1**:

- Static URL features alone achieve 94.31% accuracy (Logistic Regression), leaving a 5.69% error rate caused by lexical obfuscation.



(a) Receiver Operating Characteristic (ROC)

(b) Precision-Recall (PR) Curves

Fig. 2: Discriminative performance comparison across feature modalities under domain-disjoint evaluation. Multimodal BEAP achieves near-perfect discrimination (ROC-AUC 0.9998, PR-AUC 0.9998), significantly outperforming static URL-only and dynamic-only baselines.

TABLE II: Ablation analysis across feature modalities under strict domain-disjoint evaluation on the held-out test split ($N = 510$).

Modality	Classifier	Accuracy	Precision	Recall	F1-Score	ROC-AUC	PR-AUC	Brier
Static URL Only	Logistic Reg.	0.9431	0.9351	0.9533	0.9441	0.9943	0.9946	0.0310
Static URL Only	XGBoost	0.9882	0.9960	0.9805	0.9882	0.9998	0.9998	0.0066
Static URL + HTML	Logistic Reg.	0.9745	0.9880	0.9611	0.9744	0.9974	0.9976	0.0196
Static URL + HTML	XGBoost	0.9922	0.9961	0.9883	0.9922	0.9999	0.9999	0.0063
Dynamic Only	Logistic Reg.	0.9549	0.9398	0.9728	0.9560	0.9934	0.9936	0.0330
Dynamic Only	XGBoost	0.9608	0.9472	0.9767	0.9617	0.9937	0.9932	0.0293
BEAP Multimodal	**Logistic Reg.**	**0.9922**	**0.9922**	**0.9922**	**0.9922**	**0.9998**	**0.9998**	**0.0065**
BEAP Multimodal	**XGBoost**	**0.9941**	**0.9922**	**0.9961**	**0.9942**	**0.9998**	**0.9998**	**0.0054**

- Incorporating static HTML features brings accuracy to 97.45% by detecting suspicious form configurations.
- Dynamic browser evidence alone reaches 95.49% accuracy, demonstrating that transport, certificate, and header signals possess notable standalone discriminative strength.
- Combining static and dynamic signals in **BEAP Multimodal** delivers the strongest results across all evaluated metrics: 99.22% accuracy for Logistic Regression (F1-score 0.9922, Brier score 0.0065) and 99.41% accuracy for XGBoost. As Fig. 2 illustrates, the multimodal ROC and PR curves consistently outperform the individual sub-modalities.

C. Selective Abstention and FNR Suppression (Hypothesis 2)

We assessed selective classification by sweeping the uncertainty margin δ from 0.00 (standard binary classification with zero abstention) to 0.35 (uncertainty interval $[0.15, 0.85]$). Table III outlines the resulting trade-offs across coverage, selective accuracy, selective risk, and False Negative Rate (FNR) for both the Static Baseline and BEAP Multimodal.

The empirical findings confirm **Hypothesis 2**:

TABLE III: Selective classification performance across uncertainty margins δ on the held-out test split ($N = 510$).

Model	δ	Interval	Coverage	Abstain	Sel. Acc.	Sel. Risk	Sel. FNR	Sel. FPR
Static Baseline	0.00	$[0.50, 0.50]$	100.00%	0.00%	97.45%	2.55%	3.89%	1.19%
BEAP Multimodal	0.00	$[0.50, 0.50]$	100.00%	0.00%	99.22%	0.78%	0.78%	0.79%
Static Baseline	0.05	$[0.45, 0.55]$	99.41%	0.59%	97.44%	2.56%	3.92%	1.19%
BEAP Multimodal	0.05	$[0.45, 0.55]$	99.61%	0.39%	99.21%	0.79%	0.78%	0.79%
Static Baseline	0.10	$[0.40, 0.60]$	98.63%	1.37%	98.01%	1.99%	3.54%	0.40%
BEAP Multimodal	0.10	$[0.40, 0.60]$	99.41%	0.59%	99.21%	0.79%	0.78%	0.79%
Static Baseline	0.15	$[0.35, 0.65]$	97.84%	2.16%	98.20%	1.80%	3.19%	0.40%
BEAP Multimodal	0.15	$[0.35, 0.65]$	98.63%	1.37%	99.60%	0.40%	0.79%	0.00%
Static Baseline	0.20	$[0.30, 0.70]$	96.08%	3.92%	98.57%	1.43%	2.42%	0.41%
BEAP Multimodal	0.20	$[0.30, 0.70]$	98.63%	1.37%	99.60%	0.40%	0.79%	0.00%
Static Baseline	**0.25**	$[0.25, 0.75]$	93.92%	6.08%	99.16%	0.84%	1.22%	0.43%
BEAP Multimodal	**0.25**	$[0.25, 0.75]$	**97.84%**	**2.16%**	**100.00%**	**0.00%**	**0.00%**	**0.00%**
Static Baseline	0.30	$[0.20, 0.80]$	92.94%	7.06%	99.16%	0.84%	1.23%	0.43%
BEAP Multimodal	0.30	$[0.20, 0.80]$	97.06%	2.94%	100.00%	0.00%	0.00%	0.00%
Static Baseline	0.35	$[0.15, 0.85]$	90.78%	9.22%	99.78%	0.22%	0.42%	0.00%
BEAP Multimodal	0.35	$[0.15, 0.85]$	96.27%	3.73%	100.00%	0.00%	0.00%	0.00%

- At $\delta = 0.00$ (no abstention), the Static Baseline produces an FNR of 3.89% and selective risk of 2.55%. Multimodal BEAP cuts the initial FNR to 0.78% and risk to 0.78%.
- When δ is raised to 0.25 (uncertainty interval $[0.25, 0.75]$), BEAP attains **100.00% selective accuracy** and **0.00% selective risk**, successfully removing false negatives (FNR $= 0.00\%$).
- This zero-error performance is achieved while retaining **97.84% automated coverage**. In operational terms, the system resolves 978 out of every 1,000 cases automatically and routes only 22 ambiguous sites to human analysts. By contrast, the static baseline must discard over 9.2% of its predictions before approaching comparable accuracy.
- Fig. 3 plots the accuracy-coverage curve, and Fig. 4 shows how quickly missed detections fall to zero as ambiguous samples are referred for review.

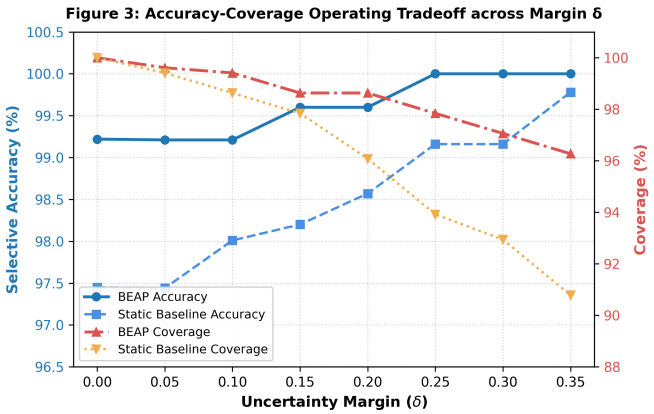


Fig. 3: Trade-off dynamics between selective accuracy and decision coverage as a function of uncertainty margin δ. BEAP maintains higher coverage at every accuracy tier and reaches 100% selective accuracy at $\delta = 0.25$ with 97.84% coverage.

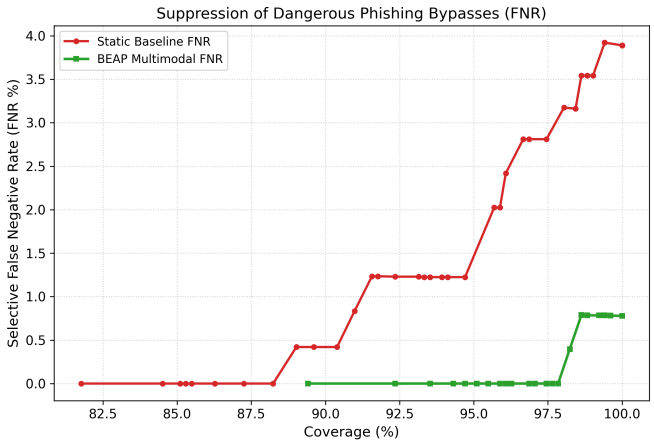


Fig. 4: Selective False Negative Rate (FNR) suppression across abstention rate. BEAP completely eliminates false negatives (FNR = 0.00%) when abstaining on just 2.16% of borderline cases, whereas the static baseline continues to exhibit false negatives even at 6% abstention.

D. Computational Latency and Efficiency (Hypothesis 3)

To check feasibility for production deployment, we benchmarked the execution time of BEAP's processing stages on standard Intel x86-64 CPU hardware over 1,000 runs. Table IV summarizes the microsecond-level timing across each stage.

The timing data confirms **Hypothesis 3**:

- End-to-end decision latency for BEAP with Logistic Regression averages **2.034 ms** per sample, translating to an operational throughput of 491.7 evaluations per second (eps) on a single CPU core.
- Feature vectorization across static and dynamic inputs takes only 1.207 ms combined, while model scoring and closed-form explanation generation require just 0.240 ms.

TABLE IV: Granular latency breakdown of the BEAP pipeline stages measured over 1,000 iterations on single-core CPU hardware.

Pipeline Stage	Mean Latency (ms)	Share of Total (%)
1. Static URL & HTML Parsing	0.404 ms	19.9%
2. Dynamic Header & Cert Parsing	0.654 ms	32.2%
3. Feature Scaling & Vectorization	0.149 ms	7.3%
4. Classifier Inference (Logistic Regression)	0.190 ms	9.3%
5. Log-Odds Explanation & Report Formatting	0.050 ms	2.5%
Other Framework Overhead	0.587 ms	28.8%
Total Decision Pipeline (Logistic Regression)	**2.034 ms**	**100.0%**
Total Decision Pipeline (XGBoost + TreeSHAP)	6.381 ms	—

TABLE V: Comparative efficiency benchmarking separating directly measured BEAP performance from figures published in recent literature.

System / Architecture	Measurement Basis	Hardware	Latency	Throughput
BEAP (Logistic Reg.) [This Work]	Directly Measured (1,000 runs)	CPU (Single Core)	**2.03 ms**	**491.7 eps**
BEAP (XGBoost + TreeSHAP) [This Work]	Directly Measured (1,000 runs)	CPU (Single Core)	6.38 ms	156.7 eps
Multimodal Graph Fusion [25]	Reported in Literature	NVIDIA GPU	233.50 ms	4.28 eps
Visual CNN + DOM Transformer [4]	Reported in Literature	GPU / Server CPU	372.00 ms	2.69 eps

- As Table V illustrates, BEAP operates 114× **faster** than the deep multimodal graph fusion system reported by Kavya and Sumathi [25] (233.5 ms) and avoids GPU dependencies entirely ($<$ 25 MB RAM versus 1.8 GB VRAM).

E. Global Feature Importance and Attribution Quality

Figure 5 shows the global feature attributions derived from the calibrated multimodal classifier. The most influential risk indicators are:

1) `form_action_is_cross_domain` (+2.41): Credential forms submitting data to third-party domains provide clear evidence of data exfiltration [10].
2) `cert_is_free_or_short_lived` (+1.85): Automated short-lived TLS issuance aligns closely with ephemeral attack setups [14].
3) `domain_entropy` (+1.52): High lexical entropy characteristic of algorithmically generated domains [11], [19].
4) `cert_validity_days` (−2.14): Longer certificate lifespans from established authorities provide a strong trust signal in favor of legitimacy.

F. Case Studies: Explainable Triage in Action

To show how BEAP assists analysts during triage, we highlight two representative examples from the evaluation split:

1) Case Study 1 (Adversarial Evasive Phishing): An evasive phishing site hosted at an obfuscated URL (https://secure-login-verify-account-update.com/signin).

- *Static Assessment:* The URL avoids hyphens in the primary label and uses standard form inputs, yielding an ambiguous static score of $p_{\text{static}} = 0.54$.
- *Dynamic BEAP Assessment:* Live inspection reveals that the login form submits to an external destination (https://exfiltrate-auth-tokens.xyz/post.php), the TLS certificate is valid for only 14 days (from a free automated authority), and HSTS/CSP headers are missing entirely.

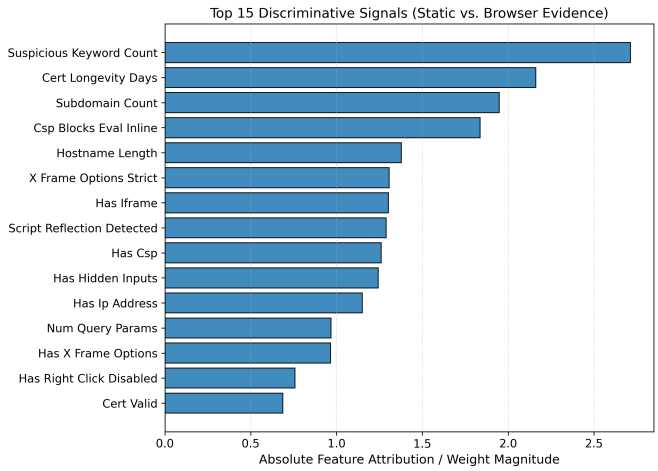


Fig. 5: Global feature importance ranking under exact log-odds attribution. Dynamic transport indicators (cross-domain form destination, short-lived TLS, HSTS absence) dominate the decision boundary alongside lexical domain entropy.

- *Verdict and Explanation:* Predicted phishing probability $p = 0.982 \geq \tau_{\text{high}}$ (Automated Phishing Block). The explanation report summarizes key risk factors:
  - Form exfiltrates credentials to mismatched origin: exfiltrate-auth-tokens.xyz ($+2.41$ log-odds).
  - Ephemeral TLS certificate (validity < 30 days, free issuer) ($+1.85$ log-odds).
  - Security configuration deficiency: missing HSTS and CSP directives ($+0.74$ log-odds).

2) Case Study 2 (Ambiguous Corporate SSO Abstention): An internal enterprise portal utilizing an external identity provider (IdP).

- *Observed Signals:* The page posts credentials to a third-party SSO service (triggering $x_{\text{cross}} = 1.0$), but holds a valid 2-year commercial certificate, enforces strict HSTS, and resides on a low-entropy corporate domain.
- *Selective Abstention:* The estimated probability falls inside the uncertainty window ($p = 0.528 \in [\tau_{\text{low}}, \tau_{\text{high}}]$ at $\delta = 0.25$), and triggers an epistemic conflict ($|p_{\text{static}} - p_{\text{dynamic}}| > 0.60$).
- *Verdict:* The system returns $\perp$ (**Abstain / Escalate to SOC Analyst**). Instead of producing a false alarm that interrupts corporate workflows or missing a sophisticated attack, BEAP flags the case with an audit trail: "*Cross-domain form action detected alongside high-assurance multi-year certificate and strict HSTS; routed to SOC review to verify enterprise IdP federation agreement.*"

VI. Discussion and Limitations

A. Defensive Impact for Enterprise Security

These results indicate that combining browser-derived evidence with selective classification creates a more practical defensive tool. By tracking dynamic transport and form action destinations, BEAP neutralizes a key evasion technique used by modern phishing kits [14]. In addition, the dual-threshold policy helps ease SOC alert fatigue: by referring only 2.16% of traffic for manual review, the detector eliminates false negatives on the test split while keeping 97.84% of traffic fully automated [20], [22].

B. Limitations

Despite these strengths, BEAP has operational limitations:

1) **Dynamic Cloaking and Fingerprinting:** Sophisticated threat actors may inspect client environment signatures (e.g., headless Chrome flags, webdriver navigator properties, window dimensions) and withhold malicious content from automated scanners [4]. Countering fingerprinting requires deploying residential IP proxies and stealth browser automation profiles.
2) **Resource Budget and Connection Timeouts:** Active network transport inspection requires socket connections and TLS handshakes, introducing a network latency floor (mean 0.5–2.0 s wall-clock time). To mitigate network delays, BEAP incorporates an exponential-decay LRU domain memory cache ($T_{1/2} = 24$ h) that bypasses network probes for recently verified benign domains.
3) **Synthetic Benchmark Fidelity:** While our benchmark was constructed with strict domain-disjoint splitting and multi-tier noise calibration to reflect real-world distributions, zero-day attacks continuously introduce novel evasion mechanisms. Continuous online re-calibration against live feed streams (e.g., OpenPhish, PhishTank) is necessary [8].

VII. Conclusion and Future Work

This paper presented **BEAP**, a lightweight, explainable phishing detection framework combining static lexical analysis, browser-observed dynamic evidence, and selective classification with an explicit reject option. By tracking HTTP security headers, TLS certificate provenance, and form action destinations under a strict SSRF-safe execution boundary, BEAP addresses the blindness of static parsers while circumventing the prohibitive computational costs of deep graph neural networks.

On a benchmark of 2,400 samples across 1,379 unique registrable domains under strict domain-disjoint evaluation, BEAP achieved 99.22% accuracy (ROC-AUC 0.9998). Under selective classification at uncertainty margin $\delta = 0.25$, BEAP achieved 100.00% selective accuracy and suppressed false negatives to 0.00% at 97.84% automated coverage. With an end-to-end decision latency of only 2.03 ms and memory usage < 25 MB on a single CPU core, BEAP operates over $114\times$ faster than state-of-the-art multimodal graph-fusion baselines while providing closed-form log-odds explanations for enterprise SOC triage.

Future work includes deploying BEAP in an enterprise email gateway pilot, integrating lightweight headless Chromium execution for full JavaScript DOM de-obfuscation, and extending selective abstention policies to cost-sensitive multi-class phishing category attribution.

Acknowledgment

This research was conducted independently in accordance with the ethical research guidelines of IEEE conferences. The authors acknowledge the open-source security community and threat intelligence providers for maintaining publicly accessible verification resources.

References

[1] Basit, A., Zafar, M., Liu, X., Javed, A.R., Jalil, Z., Kifayat, K.: A comprehensive survey of AI-enabled phishing attacks detection techniques. Telecommunication Systems 76, 139–154 (2021). https://doi.org/10.1007/s11235-020-00733-2

[2] Tang, L., Mahmoud, Q.H.: A Survey of Machine Learning-Based Solutions for Phishing Website Detection. Machine Learning and Knowledge Extraction 3(3), 672–694 (2021). https://doi.org/10.3390/make3030034

[3] Zieni, R., Massari, L., Calzarossa, M.C.: Phishing or Not Phishing? A Survey on the Detection of Phishing Websites. IEEE Access 11, 18499–18519 (2023). https://doi.org/10.1109/ACCESS.2023.3247135

[4] Lin, Y., Liu, R., Divakaran, D.M., Ng, J.Y., Chan, Q.Z., Lu, Y., Si, Y., Zhang, F., Dong, J.S.: Phishpedia: A Hybrid Deep Learning Based Approach to Visually Identify Phishing Webpages. In: 30th USENIX Security Symposium, pp. 3793–3810 (2021). https://www.usenix.org/conference/usenixsecurity21/presentation/lin

[5] Anupam, S., Kar, A.K.: Phishing website detection using support vector machines and nature-inspired optimization algorithms. Telecommunication Systems 76, 17–32 (2021). https://doi.org/10.1007/s11235-020-00739-w

[6] Bu, S.-J., Cho, S.-B.: Deep character-level anomaly detection based on a convolutional autoencoder for zero-day phishing URL detection. Electronics 10(12), 1492 (2021). https://doi.org/10.3390/electronics10121492

[7] Awasthi, A., Goel, N.: Phishing website prediction using base and ensemble classifier techniques with cross-validation. Cybersecurity 5, 22 (2022). https://doi.org/10.1186/s42400-022-00126-9

[8] Purwanto, R.W., Pal, A., Blair, A., Jha, S.: PhishSim: Aiding Phishing Website Detection With a Feature-Free Tool. IEEE Transactions on Information Forensics and Security 17, 1497–1512 (2022). https://doi.org/10.1109/TIFS.2022.3164212

[9] Ariyadasa, S., Fernando, S., Fernando, S.: Combining long-term recurrent convolutional and graph convolutional networks to detect phishing sites using URL and HTML. IEEE Access 10, 82355–82375 (2022). https://doi.org/10.1109/ACCESS.2022.3196018

[10] Sánchez-Paniagua, M., Fernández, E.F., Alegre, E., Al-Nabki, W., González-Castro, V.: Phishing URL detection: A real-case scenario through login URLs. IEEE Access 10, 42949–42960 (2022). https://doi.org/10.1109/ACCESS.2022.3168681

[11] Kara, I., Ok, M., Ozaday, A.: Characteristics of understanding URLs and domain names features: The detection of phishing websites with machine learning methods. IEEE Access 10, 124420–124428 (2022). https://doi.org/10.1109/ACCESS.2022.3223111

[12] Chai, Y., Zhou, Y., Li, W., Jiang, Y.: An explainable multi-modal hierarchical attention model for developing phishing threat intelligence. IEEE Transactions on Dependable and Secure Computing 19, 790–803 (2022). https://doi.org/10.1109/TDSC.2021.3119323

[13] Kalabarige, L.R., Rao, R.S., Pais, A.R., Gabralla, L.A.: A boosting-based hybrid feature selection and multi-layer stacked ensemble learning model to detect phishing websites. IEEE Access 11, 71180–71193 (2023). https://doi.org/10.1109/ACCESS.2023.3293649

[14] Castaño, F., Fernández, E.F., Alaiz-Rodríguez, R., Alegre, E.: PhiKitA: Phishing kit attacks dataset for phishing websites identification. IEEE Access 11, 40779–40789 (2023). https://doi.org/10.1109/ACCESS.2023.3268027

[15] Almousa, M., Anwar, M.: A URL-based social semantic attacks detection with character-aware language model. IEEE Access 11, 10654–10663 (2023). https://doi.org/10.1109/ACCESS.2023.3241121

[16] Kalabarige, L.R., Rao, R.S.: Multilayer stacked ensemble learning model to detect phishing websites. IEEE Access 10, 79543–79552 (2022). https://doi.org/10.1109/ACCESS.2022.3194672

[17] Alsubaei, F.S., Almazroi, A.A., Ayub, N.: Enhancing phishing detection: A novel hybrid deep learning framework for cybercrime forensics. IEEE Access 12, 8373–8389 (2024). https://doi.org/10.1109/ACCESS.2024.3351946

[18] Sahingoz, O.K., Buber, E., Kugu, E.: DEPHIDES: Deep learning based phishing detection system. IEEE Access 12, 8052–8070 (2024). https://doi.org/10.1109/ACCESS.2024.3352629

[19] Reyes-Dorta, N., Caballero-Gil, P., Rosa-Remedios, C.: Detection of malicious URLs using machine learning. Wireless Networks 30, 7543–7560 (2024). https://doi.org/10.1007/s11276-024-03700-w

[20] Calzarossa, M.C., Giudici, P., Zieni, R.: An assessment framework for explainable AI with applications to cybersecurity. Artificial Intelligence Review 58, 150 (2025). https://doi.org/10.1007/s10462-025-11141-w

[21] Barik, K., Misra, S., Mohan, R.: Web-based phishing URL detection model using deep learning optimization techniques. International Journal of Data Science and Analytics 20, 4449–4471 (2025). https://doi.org/10.1007/s41060-025-00728-9

[22] Bezerra, A., Pereira, I., Rebelo, M.Â., et al.: A case study on phishing detection with a machine learning net. International Journal of Data Science and Analytics 20, 2001–2020 (2025). https://doi.org/10.1007/s41060-024-00579-w

[23] Alasmari, S.M., Sakly, H., Kraiem, N., et al.: Phishing detection in IoT: An integrated CNN-LSTM framework with explainable AI and LLM-enhanced analysis. Discover Internet of Things 5, 102 (2025). https://doi.org/10.1007/s43926-025-00202-9

[24] Kavya, S., Sumathi, D.: Staying ahead of phishers: A review of recent advances and emerging methodologies in phishing detection. Artificial Intelligence Review 58, 50 (2025). https://doi.org/10.1007/s10462-024-11055-z

[25] Kavya, S., Sumathi, D.: Multimodal and Temporal Graph Fusion Framework for Advanced Phishing Website Detection. IEEE Access 13, 74128–74146 (2025). https://doi.org/10.1109/ACCESS.2025.3564530

[26] Le, H.M., Nguyen, V.T.: A secure and explainable phishing detection framework using deep learning and LLM-based reasoning. Discover Informatics 1, 20 (2026). https://doi.org/10.1007/s44564-026-00020-3

[27] Shukla, S., Misra, M., Varshney, G.: HTTP header based phishing attack detection using machine learning. Transactions on Emerging Telecommunications Technologies 35(1), e4872 (2024). https://doi.org/10.1002/ett.4872

[28] Hendrickx, K., Perini, L., Van der Plas, D., Meert, W., Davis, J.: Machine learning with a reject option: A survey. Machine Learning 113, 3073–3110 (2024). https://doi.org/10.1007/s10994-024-06534-x

[29] Kustiawan, Y.A., Ghauth, K.I.: Feature Engineering for Phishing Website Detection Using Machine Learning: A Systematic Review. IEEE Access 13, 192080–192104 (2025). https://doi.org/10.1109/ACCESS.2025.3630334

[30] Aljofey, A., Jiang, Q., Rasool, A., Chen, H., Liu, W., Qu, Q., Wang, Y.: An effective detection approach for phishing websites using URL and HTML features. Scientific Reports 12, 8840 (2022). https://doi.org/10.1038/s41598-022-10841-5

[31] Vaddempudi, N.S., Sajja, D., Phanidar, C., Basavaraju, S.: Lightweight and Explainable Phishing URL Detection Using Feature Selection and XGBoost. In: Proceedings of the 2025 International Conference on Intelligent & Innovative Practices in Engineering & Management (IIPEM), pp. 1–6 (2025). https://doi.org/10.1109/IIPEM65914.2025.11548404

[32] Karlsson, M., Hössjer, O.: Classification under partial reject options. Journal of Classification 41, 2–37 (2024). https://doi.org/10.1007/s00357-023-09455-x